Crypto ETL Pipeline

Complete Low-Level Project Handbook

A detailed end-to-end documentation of how the project evolved from running a single Python file to a Dockerized and Airflow-orchestrated data engineering pipeline.

Area	Tools used	Purpose
Source	CoinGecko API	Provides cryptocurrency market data
Extract	Python, requests, pandas	Calls API and stores raw data
Transform	pandas	Validates schema, cleans fields, writes processed file
Load	SQLAlchemy, psycopg2, PostgreSQL	Loads current and historical tables
Config	.env, python-dotenv	Keeps DB credentials outside code
Observability	logging	Writes console and file logs
Dashboard	Power BI	Provides KPIs, charts, slicers, drill-through
Automation	Windows Task Scheduler, Airflow	Runs and orchestrates pipeline
Packaging	Docker	Makes runtime reproducible

Table of Contents

1. Project Evolution from Beginning to Current State

The project started with the basic question of how to run a .py file. It then grew through multiple stages:
first Python script execution, dependency fixing, API extraction, transformation, database loading, logging, environment variables, Power BI dashboarding, Task Scheduler automation, Docker containerization, and Airflow orchestration.

1.1 Timeline of the project

1. Confirmed Python was installed using python --version.

2. Navigated to the project scripts folder using cd.

3. Ran extract.py and hit a pandas import error.

4. Diagnosed pip vs python environment mismatch.

5. Installed pandas into the correct Python environment.

6. Successfully called the CoinGecko API and printed crypto data.

7. Created transform.py to keep relevant columns and clean the data.

8. Installed PostgreSQL and created crypto_db.

9. Created load.py to load transformed data into PostgreSQL.

10. Fixed DB connection issues including special characters in password.

11. Added run_pipeline.py to run extract, transform, load together.

12. Added centralized logging through logger_config.py.

13. Added incremental-style history tracking with batch_id and load_timestamp.

14. Moved DB credentials into .env.

15. Added Power BI dashboard with overview and drill-through pages.

16. Automated runs through Windows Task Scheduler.

17. Dockerized the ETL pipeline and verified it updated PostgreSQL.

18. Moved to Airflow using Docker Compose because native Windows Airflow failed.

19. Created a single-task Airflow DAG.

20. Upgraded Airflow DAG into three tasks: extract_task, transform_task, load_task.

1.2 Final architecture

```
CoinGecko API
    -> extract.py
    -> data/crypto_raw.csv
    -> transform.py
    -> data/crypto_transformed.csv
    -> load.py
    -> PostgreSQL
        - crypto_prices_current
        - crypto_prices_history
    -> Power BI dashboard

Orchestration:
    Airflow DAG: extract_task -> transform_task -> load_task

Packaging:
    Docker image: crypto-pipeline
```

2. Local Python and Command-Line Setup

2.1 Check Python version

```
python --version
```

This confirmed Python was installed and available in PowerShell. Your version was Python 3.10.11.

2.2 Navigate to scripts folder

```
cd C:\Users\guggi\OneDrive\Desktop\crypto-data-pipeline\scripts
```

cd means change directory. This moved PowerShell into the folder that contained extract.py, transform.py, load.py, and run_pipeline.py.

2.3 Run a Python file

```
python extract.py
```

This executes extract.py using the Python interpreter resolved by the python command.

2.4 List files

```
ls
```

ls lists files in the current folder. You used it to confirm the scripts existed.

2.5 PowerShell note

Running run_pipeline.py directly failed because PowerShell does not run current directory scripts by default. The reliable command is:

```
python run_pipeline.py
```

3. Environment and Package Problem

3.1 Error

```
ModuleNotFoundError: No module named 'pandas'
```

The script could run, but Python could not import pandas.

3.2 Why pip install pandas looked confusing

```
Requirement already satisfied: pandas in c:\users\guggi\anaconda3\lib\site-packages
```

This meant pandas was installed in Anaconda, but the script was running with another Python interpreter.

3.3 Correct fix

```
python -m pip install pandas
```

python -m pip forces pip to install packages into the same Python interpreter that runs your script.

3.4 Other packages used

```
pandas
requests
sqlalchemy
psycopg2-binary
python-dotenv
```

Package	Used for
pandas	DataFrames, CSV read/write, transformations
requests	API requests
sqlalchemy	Database engine and SQL execution
psycopg2-binary	PostgreSQL driver
python-dotenv	Loading .env values

4. Extract Step

4.1 What extract.py does

The extract step calls the CoinGecko API, validates the response, converts JSON to a DataFrame, and writes raw data to crypto_raw.csv. This is the raw landing layer.

4.2 Final extract.py

```
import requests
import pandas as pd
import os
import logging
import time

from logger_config import setup_logger

setup_logger()


def extract_data():
    try:
        logging.info("Starting extract step")

        url = "https://api.coingecko.com/api/v3/coins/markets"
        params = {
            "vs_currency": "usd",
            "order": "market_cap_desc",
            "per_page": 50,
            "page": 1,
            "sparkline": False
        }

        max_retries = 3
        retry_delay = 5
        response = None

        for attempt in range(1, max_retries + 1):
            try:
                logging.info(f"API request attempt {attempt}")
                response = requests.get(url, params=params, timeout=15)
                response.raise_for_status()
                break
            except requests.exceptions.RequestException as e:
                logging.warning(f"Attempt {attempt} failed: {e}")
                if attempt == max_retries:
                    raise
                time.sleep(retry_delay)

        data = response.json()

        if not data:
            raise ValueError("API returned empty data.")

        df = pd.DataFrame(data)

        if df.empty:
            raise ValueError("Extracted DataFrame is empty.")

        os.makedirs("../data", exist_ok=True)
        df.to_csv("../data/crypto_raw.csv", index=False)

        logging.info("Raw data saved successfully to ../data/crypto_raw.csv")
        logging.info(f"Extracted {len(df)} records")

    except requests.exceptions.RequestException as e:
        logging.error(f"API request failed: {e}")
        raise
    except Exception as e:
        logging.error(f"Unexpected error in extract step: {e}")
        raise


if __name__ == "__main__":
    extract_data()
```

4.3 Line-by-line explanation

Code or concept	Explanation
import requests	Imports the HTTP library used to call CoinGecko.

Code or concept	Explanation
import pandas as pd	Imports pandas for DataFrame creation and CSV output.
setup_logger()	Initializes logging before the step runs.
url	Stores the CoinGecko endpoint.
params	Controls USD currency, top 50 records, market cap ordering, and no sparkline.
max_retries = 3	Allows three API attempts before failure.
retry_delay = 5	Waits five seconds between attempts.
requests.get(..., timeout=15)	Calls API and prevents the request from hanging forever.
response.raise_for_status()	Raises an exception for HTTP errors.
response.json()	Converts response body to Python objects.
pd.DataFrame(data)	Converts list of dictionaries into tabular data.
df.empty	Prevents empty data from continuing downstream.
os.makedirs('../data', exist_ok=True)	Creates output folder if missing.
to_csv(..., index=False)	Writes raw file without pandas index.
raise	Propagates failures to Airflow or run_pipeline.py.

5. Transform Step

5.1 What transform.py does

The transform step validates the raw schema, selects useful columns, standardizes fields, removes bad rows, removes duplicate coins, and writes crypto_transformed.csv.

5.2 Final transform.py

```
import pandas as pd
import logging

from logger_config import setup_logger

setup_logger()


def transform_data():
    try:
        logging.info("Starting transform step")

        df = pd.read_csv("../data/crypto_raw.csv")

        if df.empty:
            raise ValueError("Raw input file is empty.")

        required_columns = [
            "id",
            "symbol",
            "name",
            "current_price",
            "market_cap",
            "market_cap_rank",
            "total_volume",
            "high_24h",
            "low_24h",
            "price_change_percentage_24h",
            "circulating_supply",
            "last_updated"
        ]

        missing_columns = [col for col in required_columns if col not in df.columns]
        if missing_columns:
            raise ValueError(f"Missing required columns: {missing_columns}")

        df = df[required_columns].copy()

        df["last_updated"] = pd.to_datetime(df["last_updated"], errors="coerce")
```

```
        df["symbol"] = df["symbol"].str.upper().str.strip()
        df["name"] = df["name"].astype(str).str.strip()

        df = df.dropna(subset=["id", "symbol", "name", "current_price", "last_updated"])
        df = df.drop_duplicates(subset=["id"])

        if df.empty:
            raise ValueError("Transformed DataFrame is empty after cleaning.")

        df.to_csv("../data/crypto_transformed.csv", index=False)

        logging.info("Transformed data saved successfully to ../data/crypto_transformed.csv")
        logging.info(f"Transformed row count: {len(df)}")

    except FileNotFoundError:
        logging.error("Input file ../data/crypto_raw.csv not found")
        raise
    except Exception as e:
        logging.error(f"Unexpected error in transform step: {e}")
        raise


if __name__ == "__main__":
    transform_data()
```

5.3 Line-by-line explanation

Code or concept	Explanation
pd.read_csv('../data/crypto_raw.csv')	Reads the raw extract file.
if df.empty	Fails fast if raw input has no data.
required_columns	Defines expected API schema.
missing_columns list	Detects source schema changes.
df[required_columns].copy()	Keeps only selected fields and avoids chained assignment issues.
pd.to_datetime(..., errors='coerce')	Converts timestamp strings; invalid ones become null.
str.upper().str.strip()	Normalizes symbols like btc to BTC and removes spaces.
astype(str).str.strip()	Ensures names are strings and clean.
dropna(subset=[...])	Removes rows missing critical fields.
drop_duplicates(subset=['id'])	Keeps one row per coin in a snapshot.
to_csv(...)	Writes processed output for the load step.

6. PostgreSQL Setup, Schema, and Validation

6.1 Create database

```
CREATE DATABASE crypto_db;
```

crypto_db is the PostgreSQL database used by the pipeline.

6.2 History table

```
DROP TABLE IF EXISTS crypto_prices_history;

CREATE TABLE crypto_prices_history (
    id TEXT NOT NULL,
    symbol TEXT NOT NULL,
    name TEXT NOT NULL,
    current_price DOUBLE PRECISION,
    market_cap BIGINT,
    market_cap_rank INT,
    total_volume DOUBLE PRECISION,
    high_24h DOUBLE PRECISION,
    low_24h DOUBLE PRECISION,
    price_change_percentage_24h DOUBLE PRECISION,
    circulating_supply DOUBLE PRECISION,
    last_updated TIMESTAMP,
    batch_id TEXT NOT NULL,
```

```
    load_timestamp TIMESTAMP NOT NULL,
    PRIMARY KEY (id, batch_id)
);
```

The primary key (id, batch_id) allows the same coin across different runs but prevents duplicate rows for the same coin in the same batch.

6.3 Current table

```
DROP TABLE IF EXISTS crypto_prices_current;

CREATE TABLE crypto_prices_current (
    id TEXT PRIMARY KEY,
    symbol TEXT NOT NULL,
    name TEXT NOT NULL,
    current_price DOUBLE PRECISION,
    market_cap BIGINT,
    market_cap_rank INT,
    total_volume DOUBLE PRECISION,
    high_24h DOUBLE PRECISION,
    low_24h DOUBLE PRECISION,
    price_change_percentage_24h DOUBLE PRECISION,
    circulating_supply DOUBLE PRECISION,
    last_updated TIMESTAMP,
    batch_id TEXT NOT NULL,
    load_timestamp TIMESTAMP NOT NULL
);
```

The current table holds only one latest row per coin. id is the primary key.

6.4 Indexes

```
CREATE INDEX idx_crypto_history_symbol
ON crypto_prices_history(symbol);

CREATE INDEX idx_crypto_history_load_timestamp
ON crypto_prices_history(load_timestamp);

CREATE INDEX idx_crypto_history_market_cap_rank
ON crypto_prices_history(market_cap_rank);

CREATE INDEX idx_crypto_current_symbol
ON crypto_prices_current(symbol);

CREATE INDEX idx_crypto_current_market_cap_rank
ON crypto_prices_current(market_cap_rank);
```

Indexes improve filtering and sorting performance for common dashboard and validation queries.

6.5 Validation queries

```
SELECT * FROM crypto_prices_current LIMIT 10;
SELECT COUNT(*) FROM crypto_prices_current;
SELECT COUNT(*) FROM crypto_prices_history;
SELECT MAX(load_timestamp) FROM crypto_prices_current;

SELECT symbol, current_price, load_timestamp
FROM crypto_prices_history
WHERE symbol = 'BTC'
ORDER BY load_timestamp;

SELECT batch_id, COUNT(*)
FROM crypto_prices_history
GROUP BY batch_id
ORDER BY COUNT(*) DESC;
```

7. Load Step

7.1 Final load.py

```
import os
import logging
import uuid
from pathlib import Path
from urllib.parse import quote_plus
```

```
import pandas as pd
from dotenv import load_dotenv
from sqlalchemy import create_engine, text
from sqlalchemy.exc import SQLAlchemyError

from logger_config import setup_logger

setup_logger()

env_path = Path(__file__).resolve().parent.parent / ".env"
load_dotenv(env_path)


def load_to_postgres():
    try:
        logging.info("Starting load step")

        df = pd.read_csv("../data/crypto_transformed.csv")

        if df.empty:
            raise ValueError("Transformed input file is empty.")

        batch_id = str(uuid.uuid4())
        load_timestamp = pd.Timestamp.now()

        df["batch_id"] = batch_id
        df["load_timestamp"] = load_timestamp

        df = df.drop_duplicates(subset=["id", "batch_id"])

        username = os.getenv("DB_USER")
        password_raw = os.getenv("DB_PASSWORD")
        host = os.getenv("DB_HOST")
        port = os.getenv("DB_PORT")
        database = os.getenv("DB_NAME")

        if not all([username, password_raw, host, port, database]):
            raise ValueError(
                "Missing one or more DB environment variables. "
                "Check .env file and variable names."
            )

        password = quote_plus(password_raw)

        engine = create_engine(
            f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}"
        )

        with engine.begin() as connection:
            df.to_sql(
                "crypto_prices_history",
                con=connection,
                if_exists="append",
                index=False
            )

            connection.execute(text("DELETE FROM crypto_prices_current"))

            df.to_sql(
                "crypto_prices_current",
                con=connection,
                if_exists="append",
                index=False
            )

        with engine.connect() as connection:
            history_count = connection.execute(
                text("SELECT COUNT(*) FROM crypto_prices_history")
            ).scalar()

            current_count = connection.execute(
                text("SELECT COUNT(*) FROM crypto_prices_current")
            ).scalar()

        logging.info("Data appended to crypto_prices_history")
        logging.info("Data refreshed in crypto_prices_current")
        logging.info(f"Loaded row count: {len(df)}")
        logging.info(f"Batch ID: {batch_id}")
        logging.info(f"History table row count: {history_count}")
        logging.info(f"Current table row count: {current_count}")

    except FileNotFoundError:
```

```
        logging.error("Input file ../data/crypto_transformed.csv not found")
        raise
    except SQLAlchemyError as e:
        logging.error(f"Database error: {e}")
        raise
    except Exception as e:
        logging.error(f"Unexpected error in load step: {e}")
        raise


if __name__ == "__main__":
    load_to_postgres()
```

7.2 Line-by-line explanation

Code or concept	Explanation
Path(__file__).resolve().parent.parent / '.env'	Finds .env in the project root.
load_dotenv(env_path)	Loads credentials into environment variables.
pd.read_csv('../data/crypto_transformed.csv')	Reads processed file.
if df.empty	Prevents empty loads.
uuid.uuid4()	Creates unique batch ID.
pd.Timestamp.now()	Creates load timestamp.
df['batch_id']	Adds lineage to every row.
df['load_timestamp']	Adds batch load time to every row.
drop_duplicates(['id','batch_id'])	Prevents duplicate coin rows within same batch.
os.getenv(...)	Reads DB config from .env.
quote_plus(password_raw)	Encodes special characters in password.
create_engine(...)	Creates database connection engine.
engine.begin()	Runs DB writes inside a transaction.
to_sql(history, append)	Adds new snapshot to history.
DELETE FROM crypto_prices_current	Clears current table without dropping constraints.
to_sql(current, append)	Loads latest snapshot.
SELECT COUNT(*)	Validates row counts after load.

We changed from if_exists='replace' to DELETE + append for the current table because replace drops the table and would remove primary keys and indexes.

8. Logging, .env, and Orchestration Script

8.1 logger_config.py

```
import logging
import os

def setup_logger():
    os.makedirs("../logs", exist_ok=True)

    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s - %(levelname)s - %(message)s",
        handlers=[
            logging.FileHandler("../logs/pipeline.log"),
            logging.StreamHandler()
        ]
    )
```

This centralizes logging. Logs go to ../logs/pipeline.log and also appear in the terminal.

8.2 .env

```
DB_USER=postgres
```

```
DB_PASSWORD=your_actual_password
DB_HOST=host.docker.internal
DB_PORT=5432
DB_NAME=crypto_db
AIRFLOW_UID=50000
```

For local non-Docker runs, DB_HOST can be localhost. For Docker, host.docker.internal lets the container reach PostgreSQL running on Windows.

8.3 run_pipeline.py

```
from extract import extract_data
from transform import transform_data
from load import load_to_postgres
import logging
from logger_config import setup_logger

setup_logger()

def run_pipeline():
    try:
        extract_data()
        transform_data()
        load_to_postgres()
        logging.info("Pipeline completed successfully")
    except Exception as e:
        logging.error(f"Pipeline failed: {e}")
        raise

if __name__ == "__main__":
    run_pipeline()
```

run_pipeline.py executes extract -> transform -> load. It was used manually, by Task Scheduler, and by the first Docker image command.

9. Power BI Dashboard

9.1 Connection

```
Server: localhost:5432
Database: crypto_db
```

Power BI imported crypto_prices_current and crypto_prices_history. When Power BI showed an encryption warning, we accepted local unencrypted connection because it was a local development database.

9.2 DAX measures

```
Total_Coins = COUNTROWS(ALL('crypto_prices_current'))

Avg_Price = AVERAGEX(
    ALL('crypto_prices_current'),
    'crypto_prices_current'[current_price]
)

Max_Market_Cap = MAXX(
    ALL('crypto_prices_current'),
    'crypto_prices_current'[market_cap]
)

Latest_Load = MAX('crypto_prices_current'[load_timestamp])
```

ALL() removes slicer/filter context. It prevents KPI cards like Total Coins from changing to 1 when a single symbol is selected.

9.3 Overview page

• KPI cards: Total Coins, Average Price, Max Market Cap, Latest Load.

• Top 10 market cap bar chart.

• Top gainers and top losers by 24h percent change.

• Current market snapshot table.

• Symbol and name slicers.

9.4 Coin Detail drill-through page

• Drill-through field: symbol.

• Line chart using load_timestamp and current_price.

• Selected coin KPI cards.

• History table showing batch_id and load_timestamp.

• Back button for navigation.

10. Windows Task Scheduler

Task Scheduler was the first automation method. It proved the pipeline could run without manually typing commands.

10.1 Task action settings

```
Program/script:
C:\Users\guggi\AppData\Local\Programs\Python\Python310\python.exe

Add arguments:
"C:\Users\guggi\OneDrive\Desktop\crypto-data-pipeline\scripts\run_pipeline.py"

Start in:
C:\Users\guggi\OneDrive\Desktop\crypto-data-pipeline\scripts
```

Using the full Python path avoids file association issues. Start in is important because the scripts use relative paths like ../data.

10.2 Validation

```
SELECT MAX(load_timestamp) FROM crypto_prices_current;
```

The timestamp updated after the scheduled run, confirming the task worked.

11. Docker Setup and Dockerization

11.1 What Dockerization means

Dockerization packages your code, dependencies, and runtime into a container image. It turns local scripts into a reproducible backend batch job.

11.2 requirements.txt

```
pandas
requests
sqlalchemy
psycopg2-binary
python-dotenv
```

11.3 Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "scripts/run_pipeline.py"]
```

11.4 Dockerfile explanation

Line	Meaning
FROM python:3.10-slim	Base image with Python 3.10.
WORKDIR /app	Sets working directory inside container.

Line	Meaning
COPY requirements.txt .	Copies dependency file.
RUN pip install --no-cache-dir -r requirements.txt	Installs dependencies.
COPY . .	Copies project code.
CMD ...	Runs the pipeline when the container starts.

11.5 .dockerignore

```
venv
__pycache__
*.pyc
logs
data
.env
.pbix
```

This avoids copying secrets, logs, local data, Python cache, and Power BI files into the image.

11.6 Docker commands

```
docker build -t crypto-pipeline .
```

Builds a Docker image named crypto-pipeline from the current folder.

```
docker run --env-file .env crypto-pipeline
```

Runs the container and passes .env variables into it.

```
docker images
```

Lists images and confirms crypto-pipeline exists.

```
docker ps -a
```

Shows containers, including stopped containers.

11.7 Docker issues and fixes

Issue	Cause	Fix
docker daemon not running	Docker Desktop was closed	Open Docker Desktop and run docker info.
requirements.txt not found	File missing from project root	Create requirements.txt in root folder.
DB connection fails with localhost	Container localhost is not Windows host	Use DB_HOST=host.docker.internal.

After fixing these, the Docker container executed successfully and PostgreSQL timestamp updated.

12. Airflow Setup with Docker Compose

12.1 Why Docker Compose was needed

Native Airflow in Windows failed with ModuleNotFoundError: No module named 'fcntl'. Airflow expects POSIX/Linux behavior. Docker Compose runs Airflow inside Linux containers, which fixes this.

12.2 Create Airflow folders

```
mkdir dags, plugins, config
```

```
mkdir logs
```

These folders are mounted into Airflow containers.

12.3 Download docker-compose.yaml

```
curl.exe -LfO "https://airflow.apache.org/docs/apache-airflow/3.2.1/docker-compose.yaml"
```

Command part	Meaning
curl.exe	Windows curl command.
-L	Follow redirects.
-f	Fail on HTTP errors.
-O	Save with original file name.
URL	Official Airflow Docker Compose YAML.

12.4 Add Airflow UID to .env

```
AIRFLOW_UID=50000
```

Used by the official Airflow Docker Compose setup.

12.5 Initialize and start Airflow

```
docker compose up airflow-init
```

Initializes Airflow metadata DB and creates default user.

```
docker compose up -d
```

Starts Airflow services in the background.

```
docker ps
```

Confirms Airflow containers are running.

12.6 Airflow UI

```
URL: http://localhost:8080
Username: airflow
Password: airflow
```

12.7 Project mount in docker-compose.yaml

```
- ./:/opt/airflow/project
```

This lets Airflow containers access your local project at /opt/airflow/project.

12.8 Additional dependencies in Airflow containers

```
_PIP_ADDITIONAL_REQUIREMENTS: pandas requests sqlalchemy psycopg2-binary python-dotenv
```

This installs your ETL dependencies inside Airflow containers.

12.9 Restart after compose changes

```
docker compose down

docker compose up airflow-init

docker compose up -d
```

12.10 First Airflow DAG - one task

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator

default_args = {
    "owner": "kiran",
    "depends_on_past": False,
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="crypto_etl_pipeline",
    default_args=default_args,
```

```
    description="Runs crypto ETL pipeline using Python scripts",
    schedule="@daily",
    start_date=datetime(2026, 4, 1),
    catchup=False,
    tags=["crypto", "etl", "postgres"],
) as dag:

    run_pipeline = BashOperator(
        task_id="run_crypto_pipeline",
        bash_command=(
            "cd /opt/airflow/project/scripts && "
            "python run_pipeline.py"
        ),
    )

    run_pipeline
```

This first DAG ran the full pipeline as one BashOperator task. It proved that Airflow could trigger the ETL.

12.11 Final Airflow DAG - split tasks

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator

default_args = {
    "owner": "kiran",
    "depends_on_past": False,
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="crypto_etl_pipeline_split",
    default_args=default_args,
    description="Crypto ETL pipeline split into extract, transform, and load tasks",
    schedule="@daily",
    start_date=datetime(2026, 4, 1),
    catchup=False,
    tags=["crypto", "etl", "postgres"],
) as dag:

    extract_task = BashOperator(
        task_id="extract_task",
        bash_command="cd /opt/airflow/project/scripts && python extract.py",
    )

    transform_task = BashOperator(
        task_id="transform_task",
        bash_command="cd /opt/airflow/project/scripts && python transform.py",
    )

    load_task = BashOperator(
        task_id="load_task",
        bash_command="cd /opt/airflow/project/scripts && python load.py",
    )

    extract_task >> transform_task >> load_task
```

The split DAG is more production-like because each step has independent logs, retries, and dependency tracking.

Before	After
One task runs everything	Separate extract, transform, load tasks
Retry whole pipeline	Retry only failed task
One log	Task-specific logs
Less observability	Clear graph and status per task

13. Challenges, Root Causes, and Fixes

Problem	Root cause	Fix
No module named pandas	Package installed in wrong Python environment	Use python -m pip install pandas.
PowerShell did not run run_pipeline.py	PowerShell does not run current folder scripts directly	Use python run_pipeline.py.
Malformed DB host like 1434@localhost	Password had special character @	Encode password with quote_plus.
Password authentication failed	Wrong password in script/.env	Verify in pgAdmin and update .env.
quote_from_bytes expected bytes	DB_PASSWORD was None	Fix .env path/name and validate variables.
.env became .env.txt	Windows hid file extensions	Enable extensions and rename file.
batch_id column does not exist	Existing DB schema did not include new columns	ALTER TABLE or drop/recreate table.
Docker daemon error	Docker Desktop not running	Start Docker Desktop.
Docker could not find requirements.txt	File missing from root	Create requirements.txt at project root.
Container could not reach PostgreSQL	localhost referred to container	Use host.docker.internal.
Airflow fcntl error	Native Windows lacks POSIX fcntl	Use Airflow Docker Compose.

13.1 Root-cause analysis method

1. Read the exact error message.
2. Classify the issue as code, environment, data, credentials, database, container, or orchestration.
3. Test the smallest assumption.
4. Apply the smallest correct fix.
5. Rerun the pipeline.
6. Validate with logs and SQL queries.
7. Add a preventive improvement.

14. Interview Explanation and Resume Framing

14.1 Project story

I built an end-to-end data pipeline that ingests cryptocurrency market data from CoinGecko, transforms it using pandas, stores current and historical snapshots in PostgreSQL, visualizes the data in Power BI, containerizes the ETL with Docker, and orchestrates the workflow with Airflow.

14.2 What this demonstrates

- API ingestion
- Data transformation
- Database loading
- Current/history table design
- Batch metadata with batch_id and load_timestamp
- Logging and error propagation
- Environment variable based configuration
- Power BI dashboarding
- Docker containerization
- Airflow DAG orchestration

14.3 Resume bullets

• Built an end-to-end ETL pipeline using Python, pandas, PostgreSQL, Docker, and Airflow to ingest cryptocurrency market data from a public REST API and produce analytics-ready tables.

• Implemented modular extract, transform, and load tasks with retry logic, schema validation, structured logging, and environment-based credential management.

• Designed PostgreSQL current and historical snapshot tables with batch_id and load_timestamp metadata to support incremental tracking and time-series analysis.

• Containerized the pipeline using Docker and orchestrated task execution through Airflow DAGs, enabling reproducible runtime environments and task-level monitoring.

• Built a Power BI dashboard connected to PostgreSQL with KPI cards, market-cap rankings, gainers/losers analysis, slicers, and drill-through coin-level analysis.

15. Next Evolution into Azure

The local architecture can be mapped into Azure to make the project cloud-oriented.

Current component	Azure equivalent	Purpose
data/crypto_raw.csv	ADLS Gen2 raw zone	Cloud raw data lake layer
data/crypto_transformed.csv	ADLS Gen2 processed zone	Cloud processed layer
PostgreSQL	Azure SQL DB or Azure PostgreSQL	Managed database
Airflow	ADF or Managed Airflow	Cloud orchestration
Docker image	Azure Container Registry + Container Apps	Cloud container deployment
Power BI Desktop	Power BI Service	Cloud reporting and sharing

15.1 Recommended next local upgrade

```
data/
  raw/
    crypto/
      crypto_raw.csv
  processed/
    crypto/
      crypto_transformed.csv
```

This mimics real data lake zones: raw and processed.

16. Final Readiness Checklist

• Python ETL runs manually.

• PostgreSQL tables update.

• Power BI dashboard connects to PostgreSQL.

• Task Scheduler updates timestamp.

• Docker image builds.

• Docker container updates PostgreSQL.

• Airflow UI opens at localhost:8080.

• Single-task DAG runs.

• Split DAG shows extract_task, transform_task, load_task.

• Split DAG updates PostgreSQL latest timestamp.